FreshMart Grocery Store Database Design

Part 1 - Entity-Relationship Design

Group Members: Trevor Surgeson, Aaron Sisourath, Jay Suh, Caden Sprague, Wesley Wootton, Harish Subburaj

Date: March 16, 2026

1. Requirements Gathering Process

To design the FreshMart grocery store database, our group analyzed typical grocery store operations, including inventory management, customer transactions, employee assignments, and supplier restocking. We identified key workflows for store managers (restocking, low-stock alerts), cashiers (multi-item sales), and inventory clerks (receiving deliveries).

Project Scope:
FreshMart operates multiple grocery stores sharing a centralized database. Core entities include stores, employees, products (with SKU tracking), inventory per store, suppliers, customers, detailed transactions, and restocking orders with day-specific store hours.

Supported Queries:

- List products available at specific stores
- Find low-stock products
- Show transactions by store
- View restocking history
- Retrieve products by category
- Show the detailed purchase history of a customer

2. Business Rules Used

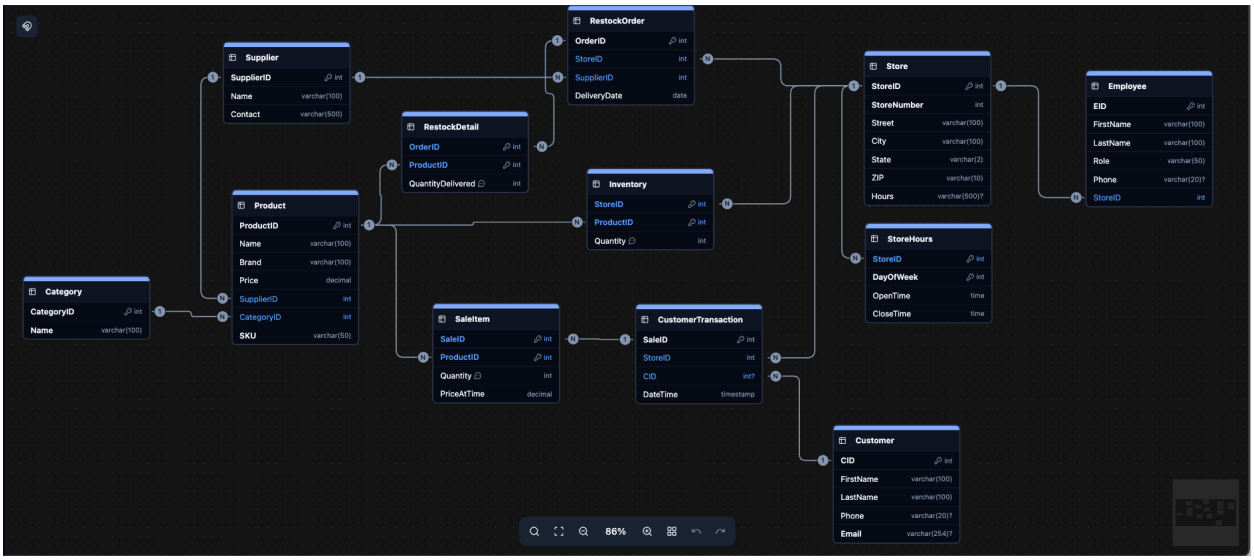
Entity Constraints:

1. Store: Unique StoreID (auto-increment), unique StoreNumber, complete address required
2. Employee: Assigned to exactly one Store; Role required (e.g., Manager, Cashier)
3. Product: Unique SKU globally; Price ≥ 0
4. Category: Predefined values only: {Produce, Meat, Dairy, Frozen, Bakery, Pantry, Cleaning, Beverages}
5. Inventory.Quantity ≥ 0 ✓ CHECK constraint
6. SaleItem.Quantity > 0 ✓ CHECK constraint
7. RestockDetail.QuantityDelivered > 0 ✓ CHECK constraint
8. Customer.Email globally unique
9. StoreHours: Exactly 7 rows per Store (DayOfWeek 1-7). OpenTime < CloseTime for each (storeID, DayOfWeek)

Relationship Constraints:

1. Store (1:N) Employee, Transaction, RestockOrder, StoreHours
2. Customer (1:N) CustomerTransaction (CID nullable = anonymous sales allowed)
3. Product (1:N) Inventory, SaleItem, RestockDetail
4. Supplier (1:N) Product, RestockOrder
5. CustomerTransaction (1:N) SaleItem
6. RestockOrder (1:N) RestockDetail

3. ER Diagram



4. Design Decisions and Alternatives

Design Choice	Rationale	Alternative
StoreHours separate table	Normalized weekly schedule (7 days × Store)	Single Hours text field in Store
CustomerTransaction (no Total)	Total computed from SaleItem	Store Total (risks inconsistency)
CID nullable	Allows anonymous sales	Require known customer only

Product.SKU unique	Global product identification	Store-specific SKUs
CHECK constraints	Enforce business rules at DB level	Application logic only
StoreNumber unique	Business identifier separate from system PK	Use StoreID as business ID

Key Assumptions:

- Anonymous sales allowed (CID nullable)
- No returns/discounts modeled
- Centralized product catalog across stores
- StoreHours uses DayOfWeek 1=Sunday...7=Saturday

5. Relational Schema Mapping

```
Table "Store" {
  "StoreID" int [pk, not null, increment]
  "StoreNumber" int [unique, not null, default: 0]
  "Street" varchar(100) [not null]
  "City" varchar(100) [not null]
  "State" varchar(2) [not null]
  "ZIP" varchar(10) [not null]
  "Hours" varchar(500)
}
```

```
Table "Employee" {
  "EID" int [pk, not null]
  "FirstName" varchar(100) [not null]
  "LastName" varchar(100) [not null]
  "Role" varchar(50) [not null]
  "Phone" varchar(20)
  "StoreID" int [not null, ref: > "Store"."StoreID"]
}

Table "Product" {
  "ProductID" int [pk, not null]
  "Name" varchar(100) [not null]
  "Brand" varchar(100) [not null]
  "Price" decimal [not null]
  "SupplierID" int [not null, ref: > "Supplier"."SupplierID"]
  "CategoryID" int [not null, ref: > "Category"."CategoryID"]
  "SKU" varchar(50) [unique, not null]

  checks {
    `Price >= 0
`
  }
```

```
}

Table "Supplier" {
  "SupplierID" int [pk, not null]
  "Name" varchar(100) [not null]
  "Contact" varchar(500) [not null]
}

Table "Category" {
  "CategoryID" int [pk, not null]
  "Name" varchar(100) [unique, not null]

  Note: 'Predefined: Produce, Meat, Dairy, Frozen, Bakery, Pantry, Cleaning,
     Beverages'
}

Table "Inventory" {
  "StoreID" int [not null, ref: > "Store"."StoreID"]
  "ProductID" int [not null, ref: > "Product"."ProductID"]
  "Quantity" int [not null, default: 0, note: '>= 0']

  Indexes {
    (StoreID, ProductID) [pk]
  }
```

```
  Note: 'Composite PK(StoreID, ProductID)'

  checks {
    `Quantity>=0`
  }
}

Table "Customer" {
  "CID" int [pk, not null]
  "FirstName" varchar(100) [not null]
  "LastName" varchar(100) [not null]
  "Phone" varchar(20)
  "Email" varchar(254) [unique]
}

Table "CustomerTransaction" {
  "SaleID" int [pk, not null]
  "StoreID" int [not null, ref: > "Store"."StoreID"]
  "CID" int [ref: > "Customer"."CID"]
  "DateTime" timestamp [not null]
}
```

```
Table "SaleItem" {
  "SaleID" int [not null, ref: > "CustomerTransaction"."SaleID"]
  "ProductID" int [not null, ref: > "Product"."ProductID"]
  "Quantity" int [not null, note: '> 0']
  "PriceAtTime" decimal [not null]

  Indexes {
    (SaleID, ProductID) [pk]
  }

  Note: 'Composite PK(SaleID, ProductID)'

  checks {
    `Quantity>0
`
    `PriceAtTime>=0
`
  }
}

Table "RestockOrder" {
  "OrderID" int [pk, not null]
  "StoreID" int [not null, ref: > "Store"."StoreID"]
```

```
  "SupplierID" int [not null, ref: > "Supplier"."SupplierID"]
  "DeliveryDate" date [not null]
}

Table "RestockDetail" {
  "OrderID" int [not null, ref: > "RestockOrder"."OrderID"]
  "ProductID" int [not null, ref: > "Product"."ProductID"]
  "QuantityDelivered" int [not null, note: '> 0']

  Indexes {
    (OrderID, ProductID) [pk]
  }

  Note: 'Composite PK(OrderID, ProductID)'

  checks {
    `QuantityDelivered>0
`
  }
}

Table "StoreHours" {
  "StoreID" int [not null, ref: > "Store"."StoreID"]
```

```
  "DayOfWeek" int [not null]
  "OpenTime" time [not null]
  "CloseTime" time [not null]

  Indexes {
    (StoreID, DayOfWeek) [pk]
  }
}
```

Constraints NOT expressible in Relational Schema:

1. Each Store has ≥1 Manager (Role='Manager') → Application logic
2. Each CustomerTransaction has ≥1 SaleItem → Application logic
3. Each RestockOrder has ≥1 RestockDetail → Application logic
4. StoreHours has exactly 7 rows per Store (1-7 days) → Application logic
5. Transaction total = Σ(SaleItem.Quantity × PriceAtTime) → View/computed column
6. Inventory never goes negative after sales/restocks → Transaction logic